

SISTEMA Y GESTIÓN DE DATA

MANUAL DEL PROGRAMADOR

DETECTOR MÁGICO

Autores:

Andrés Caiza

Frank Cueva

Anderson Pachacama

Asignatura:

Introducción a la Inteligencia Artificial

Docente:

Ing. Danny Páez

Período académico:

2026

Contenido

SISTEMA Y GESTIÓN DE DATA	1
MANUAL DEL PROGRAMADOR	1
DETECTOR MÁGICO	1
1. INTRODUCCIÓN.....	5
2. OBJETIVO DEL SISTEMA	5
2.1 Objetivo general	5
2.2 Objetivos específicos.....	5
2.3 DESCRIPCIÓN GENERAL DEL SISTEMA.....	6
2.4. Nombre del sistema	6
2.5. Descripción.....	6
2.6. Funcionamiento general.....	6
2.7. Arquitectura general.....	6
Flujo general.....	7
3. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS	7
3.1. Python	7
3.2. TensorFlow	7
3.3. Keras	7
3.4. MobileNetV2.....	8
3.5. TensorFlow.js	8
3.6. HTML5	8
3.7. CSS3	8
3.8. JavaScript	8
3.9. Google Colab.....	9
3.10. Google Drive	9
4. ESTRUCTURA DEL PROYECTO	9
4.1. Archivo index.html.....	9
4.2. Archivo style.css	10
4.3. Archivo app.js.....	10
4.4. Carpeta del modelo	10
Archivos .bin.....	11
4.5. Relación entre los archivos	11
5. Configuración del entorno	11
5.1 Herramientas utilizadas	12
5.2 Preparación del entorno.....	12

5.3 Conexión con Google Drive	12
6. Preparación y organización del dataset	12
6.1 Estructura del dataset	13
6.2 Categorías	13
6.3 Distribución de imágenes	14
7. Arquitectura y entrenamiento del modelo	15
7.1 Arquitectura del modelo	15
Arquitectura resumida	15
7.2 Configuración del entrenamiento	16
7.3 Resultado del entrenamiento	16
8. Exportación del modelo para la aplicación web	16
8.1 Guardado del modelo Keras	17
8.2 Creación del SavedModel	17
8.3 Conversión a TensorFlow.js	17
9. Estructura del proyecto web	18
9.1 Descripción de los archivos	18
9.2 Funcionamiento general	18
10. Desarrollo de la interfaz HTML	19
10.1 Inclusión de TensorFlow.js	19
10.2 Área de cámara	19
10.3 Botones de control	20
10.4 Área de resultados	20
10.5 Categorías reconocidas	20
Interfaz general	21
11. Diseño de la interfaz mediante CSS	23
11.1 Configuración general	23
11.2 Contenedor principal y encabezado	23
11.3 Tarjetas de la aplicación	24
11.4 Diseño de la cámara	24
11.5 Botones	24
11.6 Presentación del resultado	25
11.7 Categorías	26
11.8 Diseño responsive	26
12. Programación de la aplicación con JavaScript	27
12.1 Configuración del modelo	27
12.2 Variables y elementos HTML	27

12.3 Carga del modelo	28
12.4 Activación de la cámara	28
12.5 Preparación de la imagen.....	29
12.6 Detección del objeto.....	30
12.7 Selección de la predicción	30
12.8 Mostrar el resultado	31
12.9 Eventos de la aplicación	31
13. Funcionamiento general del sistema	32
13.1 Flujo del sistema	32
13.2 Integración de los componentes.....	33
13.3 Proceso de clasificación.....	33
13.4 Comunicación entre entrenamiento y aplicación	34
14. Pruebas y resultados	34
14.1 Evaluación del modelo	35
14.2 Prueba individual de las categorías.....	35
14.3 Prueba de integración web.....	35
15. Conclusiones	36

1. INTRODUCCIÓN

El presente Manual del Programador describe el desarrollo, configuración y funcionamiento de **El Detector Mágico**, una aplicación web capaz de identificar diferentes objetos mediante Inteligencia Artificial.

El sistema utiliza un modelo de aprendizaje automático desarrollado mediante **Transfer Learning**, utilizando **MobileNetV2** como modelo base. El modelo fue entrenado con un conjunto de imágenes dividido en ocho categorías: caja, celular, lentes, manzana, mouse, plátano, reloj y tijeras.

Después del entrenamiento, el modelo fue evaluado utilizando un conjunto de imágenes de prueba. Posteriormente, se creó una versión del modelo preparada para ser utilizada en una aplicación web y se convirtió al formato compatible con **TensorFlow.js**.

La aplicación web permite utilizar la cámara del dispositivo para capturar imágenes en tiempo real. Estas imágenes son procesadas mediante JavaScript y enviadas al modelo de Inteligencia Artificial, que devuelve las probabilidades correspondientes a cada una de las categorías.

Este manual tiene como finalidad explicar de manera clara la estructura del proyecto, las tecnologías utilizadas, el entrenamiento del modelo, su conversión para la web y el funcionamiento de los archivos que conforman la aplicación.

2. OBJETIVO DEL SISTEMA

2.1 Objetivo general

Desarrollar una aplicación web capaz de reconocer objetos mediante Inteligencia Artificial, utilizando un modelo entrenado con Transfer Learning y ejecutándolo directamente desde el navegador mediante TensorFlow.js.

2.2 Objetivos específicos

- Preparar un conjunto de imágenes para el entrenamiento del modelo.
- Entrenar un modelo de clasificación utilizando MobileNetV2.
- Evaluar el rendimiento del modelo mediante datos de prueba.
- Preparar el modelo para su utilización en una aplicación web.
- Convertir el modelo a un formato compatible con TensorFlow.js.
- Implementar una interfaz web para utilizar la cámara del dispositivo.
- Mostrar el objeto detectado y su porcentaje de confianza.
- Permitir que el modelo reconozca ocho categorías diferentes.

2.3 DESCRIPCIÓN GENERAL DEL SISTEMA

2.4. Nombre del sistema

El Detector Mágico

2.5. Descripción

El Detector Mágico es una aplicación web que utiliza Inteligencia Artificial para identificar objetos mediante la cámara del dispositivo. El sistema fue desarrollado utilizando técnicas de **Transfer Learning** y el modelo **MobileNetV2**, permitiendo reconocer ocho categorías diferentes de objetos.

Las categorías que puede identificar el sistema son:

- Caja
- Celular
- Lentes
- Manzana
- Mouse
- Plátano
- Reloj
- Tijeras

El modelo fue entrenado utilizando un conjunto de imágenes dividido en datos de entrenamiento, validación y prueba. Posteriormente, el modelo entrenado fue convertido a un formato compatible con **TensorFlow.js**, permitiendo realizar las predicciones directamente desde el navegador.

2.6. Funcionamiento general

El funcionamiento del sistema se puede resumir en los siguientes pasos:

1. El navegador carga la aplicación web.
2. Se carga el modelo de Inteligencia Artificial.
3. El usuario activa la cámara.
4. El sistema obtiene la imagen proveniente de la cámara.
5. La imagen se redimensiona a **224 × 224 píxeles**.
6. Los valores de la imagen son normalizados para ser compatibles con el modelo.
7. TensorFlow.js procesa la imagen.
8. El modelo genera ocho probabilidades, una para cada categoría.
9. El sistema selecciona la categoría con mayor probabilidad.
10. Se muestra en pantalla el objeto detectado y su porcentaje de confianza.

2.7. Arquitectura general

El sistema está compuesto principalmente por tres elementos:

Interfaz web: desarrollada con HTML y CSS, encargada de mostrar la cámara, botones, resultados y categorías.

Lógica de la aplicación: desarrollada en JavaScript, encargada de controlar la cámara, preparar las imágenes y ejecutar las predicciones.

Modelo de Inteligencia Artificial: desarrollado y entrenado con TensorFlow/Keras y posteriormente convertido a TensorFlow.js para ejecutarse en el navegador.

Flujo general

Cámara → Imagen → Preprocesamiento → TensorFlow.js → Modelo MobileNetV2 → Predicción → Resultado en pantalla

3. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS

Para el desarrollo de **El Detector Mágico** se utilizaron diferentes tecnologías para cubrir las etapas de preparación de datos, entrenamiento del modelo de Inteligencia Artificial y desarrollo de la aplicación web.

3.1. Python

Python fue utilizado principalmente para la preparación del conjunto de datos, construcción, entrenamiento y evaluación del modelo de Inteligencia Artificial.

También se utilizó para realizar pruebas sobre el modelo antes de integrarlo en la aplicación web.

3.2. TensorFlow

TensorFlow es el framework utilizado para construir y entrenar el modelo de clasificación de imágenes.

Mediante TensorFlow se realizaron procesos como la carga del dataset, entrenamiento de la red neuronal, evaluación del modelo y exportación del modelo entrenado.

3.3. Keras

Keras se utilizó como API de alto nivel para definir la arquitectura de la red neuronal y facilitar el proceso de entrenamiento.

El modelo desarrollado utiliza una arquitectura basada en **MobileNetV2**, a la cual se añadió una capa de clasificación para las ocho categorías del proyecto.

3.4. MobileNetV2

MobileNetV2 es la arquitectura utilizada como modelo base mediante la técnica de **Transfer Learning**.

Se eligió debido a que es una arquitectura relativamente ligera y adecuada para aplicaciones donde se necesita realizar inferencia de imágenes de manera eficiente.

En el proyecto, las capas principales de MobileNetV2 se utilizaron como extractor de características y se añadió una capa final con ocho salidas correspondientes a las categorías reconocidas.

3.5. TensorFlow.js

TensorFlow.js permite ejecutar modelos de Inteligencia Artificial directamente en aplicaciones JavaScript.

En este proyecto se utilizó para cargar el modelo convertido y realizar las predicciones dentro del navegador, evitando la necesidad de enviar las imágenes de la cámara a un servidor para realizar la clasificación.

3.6. HTML5

HTML5 se utilizó para construir la estructura de la aplicación web.

Entre sus elementos se encuentran la sección de la cámara, botones de control, área de resultados, indicador de confianza y listado de categorías.

3.7. CSS3

CSS3 se utilizó para diseñar la interfaz de usuario.

Se implementaron estilos para los botones, tarjetas, cámara, barra de confianza, colores, distribución de elementos y adaptación de la aplicación a diferentes tamaños de pantalla.

3.8. JavaScript

JavaScript contiene la lógica principal de la aplicación web.

Se encarga de cargar el modelo TensorFlow.js, activar y detener la cámara, capturar las imágenes, preparar los datos de entrada, ejecutar la predicción y mostrar los resultados al usuario.

3.9. Google Colab

Google Colab fue utilizado como entorno de desarrollo para ejecutar el código Python y realizar el entrenamiento del modelo de Inteligencia Artificial.

Su utilización permitió disponer de un entorno preparado para trabajar con TensorFlow y realizar los procesos de entrenamiento y evaluación.

3.10. Google Drive

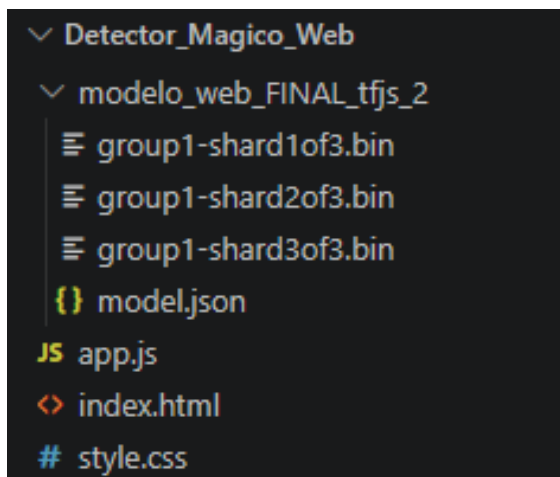
Google Drive fue utilizado para almacenar el conjunto de datos y los diferentes archivos generados durante el desarrollo.

Entre los archivos almacenados se encuentran el modelo entrenado, las clases del modelo, el SavedModel y la versión convertida a TensorFlow.js.

4. ESTRUCTURA DEL PROYECTO

La aplicación web de **El Detector Mágico** está organizada en diferentes archivos y carpetas. Cada elemento cumple una función específica dentro del sistema.

Una estructura básica del proyecto es la siguiente:



4.1. Archivo index.html

El archivo **index.html** contiene la estructura principal de la aplicación web.

En este archivo se encuentran:

- Título de la aplicación.
- Área donde se muestra la cámara.
- Botón para activar la cámara.
- Botón para realizar la detección.
- Botón para detener la cámara.
- Área donde se muestra el objeto detectado.
- Porcentaje de confianza.
- Lista de categorías reconocidas.
- Referencias a TensorFlow.js y al archivo app.js.

4.2. Archivo style.css

El archivo **style.scc** contiene los estilos visuales de la aplicación.

Se utiliza para controlar:

- Colores y fondos.
- Tamaños y tipos de letra.
- Tarjetas de información.
- Diseño de la cámara.
- Botones.
- Barra de confianza.
- Distribución de las categorías.
- Adaptación a dispositivos móviles.

De esta manera, el código HTML se mantiene separado de la presentación visual.

4.3. Archivo app.js

El archivo app.js contiene la lógica principal de la aplicación.

Entre sus funciones principales se encuentran:

- Cargar el modelo de TensorFlow.js.
- Definir las ocho categorías..
- Activar la cámara.
- Detener la cámara.
- Capturar la imagen del video.
- Preparar la imagen para el modelo.
- Ejecutar la predicción.
- Calcular la categoría con mayor probabilidad.
- Mostrar el resultado y el porcentaje de confianza.

4.4. Carpeta del modelo

La carpeta **modelo_web_FINAL_tfjs_2** contiene el modelo convertido para ser utilizado mediante TensorFlow.js.

Dentro de esta carpeta se encuentran:

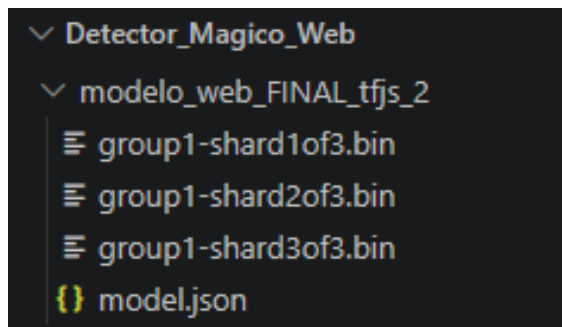
Modelo.json

Es el archivo principal del modelo. Contiene la estructura de la red neuronal y las referencias a los archivos que almacenan sus pesos.

La aplicación utiliza este archivo para cargar el modelo mediante TensorFlow.js.

Archivos .bin

Los archivos:



Contienen los pesos del modelo de Inteligencia Artificial.

Estos archivos son necesarios para que TensorFlow.js pueda reconstruir y ejecutar el modelo en el navegador.

4.5. Relación entre los archivos

El funcionamiento de los archivos puede resumirse de la siguiente manera, donde index.html utiliza los archivos para leerlos adecuadamente.

La correcta ubicación de model.json y de los archivos .bin es necesaria para que la aplicación pueda cargar el modelo correctamente.

5. Configuración del entorno

Para desarrollar y entrenar **El Detector Mágico** se utilizó **Google Colab**, debido a que permite ejecutar código Python y utilizar las bibliotecas necesarias para el entrenamiento del modelo de Inteligencia Artificial.

El proyecto utiliza principalmente **Python, TensorFlow, Keras y TensorFlow.js**.

5.1 Herramientas utilizadas

Herramienta	Función
Google Colab	Entorno para ejecutar el código de entrenamiento
Python	Lenguaje utilizado para desarrollar el modelo
TensorFlow	Framework utilizado para crear y entrenar la red neuronal
Keras	API utilizada para construir el modelo
MobileNetV2	Modelo base utilizado mediante Transfer Learning
TensorFlow.js	Permite ejecutar el modelo en el navegador
HTML	Estructura de la aplicación web
CSS	Diseño y apariencia de la aplicación
JavaScript	Funcionamiento de la aplicación y detección mediante la cámara

5.2 Preparación del entorno

El entrenamiento del modelo se realizó utilizando Google Colab. En este entorno se instalaron y configuraron las bibliotecas necesarias para trabajar con Inteligencia Artificial y posteriormente convertir el modelo para su utilización en una aplicación web.

La versión principal utilizada para el entrenamiento del modelo final fue TensorFlow: 2.20.0

Para la conversión del modelo a TensorFlow.js se utilizó el conversor correspondiente, verificando posteriormente que se generaran correctamente el archivo model.json y los archivos .bin.

5.3 Conexión con Google Drive

Los datos y modelos del proyecto se almacenaron en Google Drive. Para utilizar estos archivos desde Google Colab se realizó el montaje de la unidad.

Esto permite acceder al dataset, guardar los modelos entrenados y conservar los archivos generados durante el desarrollo.

6. Preparación y organización del dataset

El dataset utilizado para **El Detector Mágico** contiene imágenes de los objetos que el modelo debe reconocer. Las imágenes se organizaron en diferentes carpetas para realizar correctamente las etapas de entrenamiento, validación y prueba.

6.1 Estructura del dataset

El **dataset** se encuentra organizado de la siguiente manera:

```
detector_magico_dataset/  
├── train/  
│   ├── caja/  
│   ├── celular/  
│   ├── lentes/  
│   ├── manzana/  
│   ├── mouse/  
│   ├── platano/  
│   ├── reloj/  
│   └── tijeras/  
├── validation/  
│   ├── caja/  
│   ├── celular/  
│   ├── lentes/  
│   ├── manzana/  
│   ├── mouse/  
│   ├── platano/  
│   ├── reloj/  
│   └── tijeras/  
└── test/  
    ├── caja/  
    ├── celular/  
    ├── lentes/  
    ├── manzana/  
    ├── mouse/  
    ├── platano/  
    ├── reloj/  
    └── tijeras/
```

Cada carpeta representa una categoría que el modelo debe aprender a identificar.

6.2 Categorías

El modelo final trabaja con **8 categorías**:

Índice	Categoría
0	Caja
1	Celular
2	Lentes
3	Manzana
4	Mouse
5	Plátano
6	Reloj
7	Tijeras

La correspondencia entre el índice y el nombre de la categoría es importante, ya que posteriormente se utiliza para interpretar la salida del modelo.

6.3 Distribución de imágenes

Para cada categoría se utilizaron:

- **210 imágenes** para entrenamiento.
- **45 imágenes** para validación.
- **45 imágenes** para prueba.

En total se utilizaron **2400 imágenes** distribuidas entre las ocho categorías.

Aquí sí conviene poner el código utilizado para cargar las carpetas:

```
import tensorflow as tf

ruta = "/content/drive/MyDrive/detector_magico_dataset"

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

print("=" * 60)
print("CARGANDO DATASET")
print("=" * 60)

train_ds = tf.keras.utils.image_dataset_from_directory(
    ruta + "/train",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=True
)

validation_ds = tf.keras.utils.image_dataset_from_directory(
    ruta + "/validation",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

test_ds = tf.keras.utils.image_dataset_from_directory(
    ruta + "/test",
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    shuffle=False
)

class_names = train_ds.class_names

print("\n" + "=" * 60)
print("CATEGORÍAS")
print("=" * 60)

for i, clase in enumerate(class_names):
    print(i, "→", clase)

print("\nNúmero de clases:", len(class_names))
```

Resultado esperado

Al ejecutar el código se obtiene una salida similar a:

```
=====
Found 1680 files belonging to 8 classes.
Found 360 files belonging to 8 classes.
Found 360 files belonging to 8 classes.

=====
CATEGORÍAS
=====
0 → caja
1 → celular
2 → lentes
3 → manzana
4 → mouse
5 → platano
6 → reloj
7 → tijeras

Número de clases: 8
```

7. Arquitectura y entrenamiento del modelo

7.1 Arquitectura del modelo

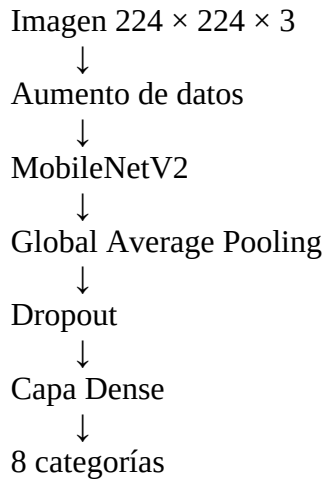
Para desarrollar el sistema se utilizó **Transfer Learning**, empleando la arquitectura **MobileNetV2** como modelo base.

MobileNetV2 permite utilizar características aprendidas previamente a partir de grandes conjuntos de imágenes. En este proyecto, sus capas principales se mantienen congeladas y se agrega una capa de clasificación para reconocer las ocho categorías del dataset.

La arquitectura utilizada está compuesta principalmente por:

1. **Entrada:** imágenes de 224×224 píxeles con 3 canales de color.
2. **Aumento de datos:** permite generar variaciones de las imágenes durante el entrenamiento.
3. **MobileNetV2:** extrae las características visuales de los objetos.
4. **Global Average Pooling:** convierte las características extraídas en un vector.
5. **Dropout:** ayuda a reducir el sobreajuste.
6. **Dense:** realiza la clasificación final en las 8 categorías.

Arquitectura resumida



7.2 Configuración del entrenamiento

El modelo se entrenó utilizando el dataset previamente preparado.

La configuración principal fue:

- **Modelo base:** MobileNetV2.
- **Tamaño de imagen:** 224×224 .
- **Número de clases:** 8.
- **Imágenes de entrenamiento:** 1680.
- **Imágenes de validación:** 360.
- **Imágenes de prueba:** 360.
- **Épocas:** 15.
- **Parámetros entrenables:** 10 248.
- **Parámetros no entrenables:** 2 257 984.

La mayor parte de los parámetros pertenecen a MobileNetV2 y permanecen congelados, mientras que la capa final se entrena para adaptar el modelo a las categorías del proyecto.

7.3 Resultado del entrenamiento

Durante el entrenamiento, el modelo alcanzó valores elevados de precisión tanto en entrenamiento como en validación.

Este resultado indica que el modelo clasificó correctamente la gran mayoría de las imágenes utilizadas para la prueba.

8. Exportación del modelo para la aplicación web

Una vez finalizado el entrenamiento y comprobado el funcionamiento del modelo, fue necesario convertirlo a un formato compatible con **TensorFlow.js** para poder utilizarlo directamente desde el navegador.

8.1 Guardado del modelo Keras

El modelo entrenado se guardó en formato **.keras** dentro de Google Drive:

```
/content/drive/MyDrive/modelo_detector_magico_FINAL.keras
```

También se guardó un archivo **.json** con los nombres de las clases:

```
/content/drive/MyDrive/clases_detector_magico_FINAL.json
```

El archivo de clases permite mantener la correspondencia entre las salidas numéricas del modelo y los nombres de los objetos.

8.2 Creación del SavedModel

Para realizar la conversión a TensorFlow.js se creó una versión del modelo destinada a inferencia web.

Durante este proceso se eliminó el componente de **Data Augmentation** utilizado únicamente durante el entrenamiento. Esto evita problemas relacionados con elementos aleatorios del modelo al momento de exportarlo.

El SavedModel generado fue:

```
/content/drive/MyDrive/modelo_saved_FINAL_WEB_2
```

La entrada del modelo es:

```
(None, 224, 224, 3)
```

y la salida:

```
(None, 8)
```

Esto significa que el modelo recibe una imagen de 224×224 píxeles y genera ocho valores correspondientes a las ocho categorías.

8.3 Conversión a TensorFlow.js

Finalmente, el SavedModel se convirtió utilizando TensorFlow.js.

La conversión generó los siguientes archivos:

modelo_web_FINAL_tfjs_2/

- model.json
- group1-shard1of3.bin
- group1-shard2of3.bin
- group1-shard3of3.bin

El archivo model.json contiene la estructura del modelo, mientras que los archivos .bin contienen los pesos necesarios para realizar las predicciones. Resultando con la conversión finalizada correctamente con código de salida 0, por lo que el modelo quedó preparado para ser utilizado desde la aplicación web mediante TensorFlow.js.

9. Estructura del proyecto web

La aplicación web está organizada en diferentes archivos y carpetas, cada uno encargado de una función específica.

La estructura utilizada es la siguiente:

El Detector Mágico/

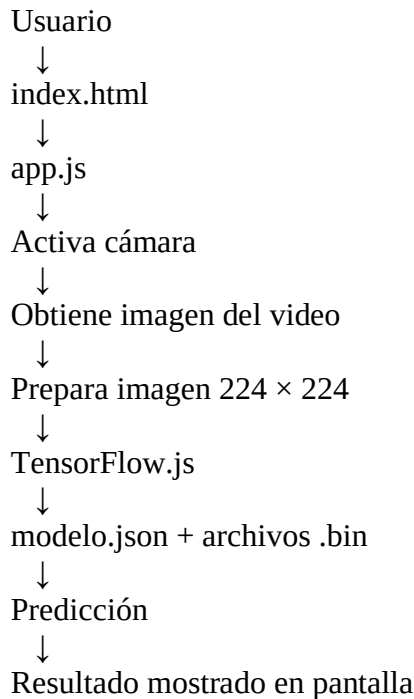
- index.html
- style.css
- app.js
- modelo_web_FINAL_tfjs_2/
 - model.json
 - group1-shard1of3.bin
 - group1-shard2of3.bin
 - group1-shard3of3.bin

9.1 Descripción de los archivos

Archivo / carpeta	Función
index.html	Contiene la estructura y los elementos visuales de la aplicación.
style.css	Contiene los estilos, colores, distribución y diseño de la interfaz.
app.js	Contiene la lógica de la aplicación, acceso a la cámara y predicción del modelo.
modelo_web_FINAL_tfjs_2/	Contiene el modelo convertido para TensorFlow.js.
model.json	Define la estructura del modelo y la ubicación de sus pesos.
group1-shard*.bin	Contienen los pesos del modelo necesarios para realizar las predicciones.

9.2 Funcionamiento general

El funcionamiento de la aplicación se puede resumir de la siguiente manera:



Esta organización permite separar la interfaz, los estilos, la lógica de programación y el modelo de Inteligencia Artificial, facilitando el mantenimiento y futuras modificaciones del sistema.

10. Desarrollo de la interfaz HTML

El archivo index.html define la estructura principal de la aplicación web **El Detector Mágico**. En él se encuentran la presentación del sistema, el estado del modelo, la cámara, los botones de control, el resultado de la detección y las categorías reconocidas.

10.1 Inclusión de TensorFlow.js

Para utilizar el modelo de Inteligencia Artificial desde el navegador se incluye la biblioteca TensorFlow.js mediante un CDN:

```
<script  
src="https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@4.22.0/dist/tf.min.  
js"></script>
```

Esta biblioteca permite cargar el modelo convertido y ejecutar las predicciones directamente en el navegador.

10.2 Área de cámara

La interfaz contiene un elemento `<video>` que permite mostrar la imagen capturada por la cámara:

```
<video id="video" autoplay playsinline></video>
```

El identificador video es utilizado posteriormente desde JavaScript para acceder a la transmisión de la cámara y obtener las imágenes que serán analizadas.

10.3 Botones de control

La aplicación dispone de tres botones principales:

```
<button id="startCamera">📷 Activar cámara</button>
```

```
<button id="detectButton" disabled>🔍 Detectar objeto</button>
```

```
<button id="stopCamera" disabled>⏏ Detener cámara</button>
```

Estos botones permiten:

- Activar la cámara.
- Ejecutar una detección.
- Detener la cámara.

El botón de detección comienza deshabilitado y se habilita cuando el modelo y la cámara están disponibles.

10.4 Área de resultados

Después de realizar una predicción, el sistema muestra el objeto identificado, su emoji y el porcentaje de confianza.

Los elementos principales son:

```
<div id="resultIcon"> ? </div>
```

```
<div id="resultName">Esperando detección...</div>
```

```
<span id="confidenceText">0%</span>
```

```
<div id="confidenceBar"></div>
```

JavaScript modifica estos elementos de acuerdo con el resultado obtenido por el modelo.

10.5 Categorías reconocidas

La interfaz también muestra las ocho categorías que puede reconocer el modelo:

- Caja
- Celular
- Lentes
- Manzana
- Mouse
- Plátano
- Reloj
- Tijeras

De esta manera, el usuario puede conocer previamente los objetos que el sistema está preparado para identificar.

Interfaz general



Resultado de la detección incluyendo las categorías que reconoce

Resultado de la detección



Esperando Detección...

Confianza

0%

Activa la cámara y coloca un objeto frente a ella.

Categorías que reconoce



Caja



Celular



Lentes



Manzana



Mouse



Plátano



Reloj



Tijeras

11. Diseño de la interfaz mediante CSS

El archivo **style.css** contiene los estilos visuales de **El Detector Mágico**. Se utiliza para definir colores, tamaños, distribución de los elementos, botones, cámara, resultados y adaptación a dispositivos móviles.

11.1 Configuración general

Se eliminan los márgenes y espacios predeterminados del navegador y se utiliza `box-sizing` para facilitar el control de las dimensiones:

```
* {  
    margin: 0;  
    padding: 0;  
    box-sizing: border-box;  
}
```

El fondo principal utiliza un degradado y la página ocupa como mínimo toda la altura de la pantalla:

```
body {  
    font-family: Arial, Helvetica, sans-serif;  
    min-height: 100vh;  
    background: linear-gradient(135deg, #667eea, #764ba2);  
    color: #222;  
}
```

11.2 Contenedor principal y encabezado

El contenido se limita a un ancho máximo para mantener una distribución ordenada:

```
.container {  
    width: 100%;  
    max-width: 1100px;  
    margin: auto;  
    padding: 20px;  
}
```

El encabezado utiliza texto centrado y color blanco:

```
header {  
    text-align: center;  
    color: white;  
    padding: 30px 20px;  
}  
  
header h1 {  
    font-size: 42px;  
    margin-bottom: 10px;  
}
```

11.3 Tarjetas de la aplicación

Las secciones principales utilizan tarjetas blancas con bordes redondeados y sombras para diferenciarlas visualmente:

```
.camera-card,  
.result-card,  
.classes-card,  
.status-card {  
  background: white;  
  border-radius: 20px;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.15);  
}
```

Estas tarjetas contienen el estado del modelo, la cámara, el resultado de la detección y las categorías.

11.4 Diseño de la cámara

El área de video se encuentra dentro de un contenedor con relación de aspecto 16:9:

```
.video-container {  
  position: relative;  
  width: 100%;  
  max-width: 800px;  
  margin: auto;  
  background: #111;  
  border-radius: 15px;  
  overflow: hidden;  
  aspect-ratio: 16 / 9;  
}
```

El video ocupa todo el espacio disponible:

```
video {  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
  display: block;  
}
```

Además, se utiliza una superposición (camera-overlay) para mostrar las esquinas que sirven como referencia visual para el área de detección.

11.5 Botones

Los botones comparten características generales como tamaño, bordes redondeados y transición:

```
.btn {  
  border: none;  
  padding: 14px 25px;  
  border-radius: 10px;  
  font-size: 16px;
```



```
    font-weight: bold;
    cursor: pointer;
    transition: 0.2s;
    color: white;
}
```

Se utilizan diferentes clases para identificar la función de cada botón:

```
.primary {
    background: #667eea;
}

.success {
    background: #28a745;
}

.danger {
    background: #dc3545;
}
```

También se establece un estilo diferente para los botones deshabilitados:

```
.btn:disabled {
    opacity: 0.45;
    cursor: not-allowed;
}
```

11.6 Presentación del resultado

El resultado de la detección se muestra de forma centrada. El nombre del objeto utiliza un tamaño de letra mayor para destacarlo:

```
.result-name {
    font-size: 35px;
    font-weight: bold;
    text-transform: capitalize;
    margin-bottom: 20px;
}
```

La barra de confianza muestra visualmente el porcentaje obtenido:

```
.progress {
    height: 18px;
    background: #e9ecef;
    border-radius: 20px;
    overflow: hidden;
}

.progress-bar {
    height: 100%;
    width: 0%;
    background: linear-gradient(90deg, #667eea, #764ba2);
    transition: width 0.4s;
}
```

JavaScript modifica posteriormente el valor de `width` de esta barra según la confianza obtenida.

11.7 Categorías

Las ocho categorías se organizan mediante CSS Grid:

```
.classes {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 15px;
}
```

Cada categoría se presenta como una tarjeta individual:

```
.class-item {
  background: #f5f5f5;
  border-radius: 15px;
  padding: 20px;
  text-align: center;
  transition: 0.2s;
}
```

11.8 Diseño responsive

Se utilizan consultas @media para adaptar la aplicación a pantallas pequeñas.

```
@media (max-width: 700px) {

  header h1 {
    font-size: 32px;
  }

  .classes {
    grid-template-columns: repeat(2, 1fr);
  }

  .btn {
    width: 100%;
  }

  .result-name {
    font-size: 28px;
  }
}
```

De esta forma, la aplicación puede utilizarse en computadores, tablets y teléfonos móviles sin perder la estructura principal de la interfaz.

12. Programación de la aplicación con JavaScript

El archivo `app.js` contiene la lógica principal de **El Detector Mágico**. Su función es cargar el modelo `TensorFlow.js`, controlar la cámara, preparar las imágenes y mostrar el resultado de la clasificación.

12.1 Configuración del modelo

Primero se establece la ubicación del modelo y las categorías que puede reconocer:

```
const MODEL_PATH = "../modelo_web_FINAL_tfjs_2/model.json";

const CLASSES = [
  "caja",
  "celular",
  "lentes",
  "manzana",
  "mouse",
  "platano",
  "reloj",
  "tijeras"
];
```

El orden de `CLASSES` debe coincidir con el orden utilizado durante el entrenamiento. Por ejemplo, la posición 0 corresponde a caja y la posición 7 corresponde a tijeras.

También se establecen los emojis que se mostrarán para cada objeto:

```
const EMOJIS = {
  caja: "📦",
  celular: "📱",
  lentes: "🕶️",
  manzana: "🍏",
  mouse: "🖱️",
  platano: "🍌",
  reloj: "🕒",
  tijeras: "✂️"
};
```

12.2 Variables y elementos HTML

Se utilizan variables para almacenar el modelo, la cámara y su estado:

```
let model = null;
let stream = null;
let cameraActive = false;
```

Además, JavaScript obtiene los elementos HTML mediante sus identificadores:

```
const video = document.getElementById("video");

const startCameraButton =
  document.getElementById("startCamera");

const detectButton =
  document.getElementById("detectButton");

const stopCameraButton =
  document.getElementById("stopCamera");
```

Esto permite que JavaScript pueda modificar la interfaz y controlar los botones de la aplicación.

12.3 Carga del modelo

La función `loadModel()` se ejecuta al iniciar la aplicación. Utiliza `tf.loadGraphModel()` para cargar el modelo convertido a TensorFlow.js:

```
async function loadModel() {
  try {
    model = await tf.loadGraphModel(MODEL_PATH);
    console.log("Modelo cargado correctamente");
  } catch (error) {
    console.error("Error al cargar el modelo", error);
  }
}
```

El archivo `model.json` contiene la estructura del modelo y los archivos `.bin` contienen sus pesos.

Cuando el modelo se carga correctamente, el botón de detección puede habilitarse cuando la cámara también se encuentre activa.

12.4 Activación de la cámara

La función `startCamera()` solicita permiso para utilizar la cámara del dispositivo mediante `getUserMedia()`:

```
stream = await navigator.mediaDevices.getUserMedia({
  video: {
    facingMode: {
      ideal: "environment"
    },
    width: {
      ideal: 1280
    }
  }
});
```

```

        },
        height: {
            ideal: 720
        }
    },
    audio: false
});

```

Después, la transmisión se asigna al elemento <video>:

```

video.srcObject = stream;

await video.play();

cameraActive = true;

```

La opción `environment` intenta utilizar la cámara trasera en dispositivos que disponen de ella.

12.5 Preparación de la imagen

Antes de enviarla al modelo, la imagen obtenida desde la cámara debe tener el formato esperado por MobileNetV2.

La función `prepareImage()` realiza este proceso:

```

function prepareImage() {
    return tf.tidy(() => {
        let image = tf.browser.fromPixels(video);

        image = tf.image.resizeBilinear(
            image,
            [224, 224]
        );

        image = image.toFloat();

        image = image.div(255.0);

        image = image.expandDims(0);

        return image;
    });
}

```

El proceso realiza cuatro pasos principales:

1. Obtiene los píxeles del video.
2. Redimensiona la imagen a 224×224 .
3. Convierte los valores a tipo float.
4. Normaliza los valores entre 0 y 1.
5. Agrega la dimensión correspondiente al lote.

El resultado tiene la forma:

```
(1, 224, 224, 3)
```

12.6 Detección del objeto

La función `detectObject()` ejecuta el modelo sobre la imagen preparada:

```
const image = prepareImage();

let prediction;

if (typeof model.executeAsync === "function") {
    prediction = await model.executeAsync(image);
} else {
    prediction = model.execute(image);
}
```

Después se obtiene la salida del modelo:

```
let output;

if (Array.isArray(prediction)) {
    output = prediction[0];
} else {
    output = prediction;
}

const probabilities = await output.data();
```

El modelo genera **8 valores de probabilidad**, uno para cada categoría.

12.7 Selección de la predicción

Se busca la categoría que tenga la probabilidad más alta:

```
let maxIndex = 0;
let maxProbability = probabilities[0];

for (let i = 1; i < probabilities.length; i++) {
    if (probabilities[i] > maxProbability) {
        maxProbability = probabilities[i];
        maxIndex = i;
    }
}

const detectedClass = CLASSES[maxIndex];
const confidence = maxProbability * 100;
```

Por ejemplo, si la posición con mayor probabilidad es 4, el resultado corresponde a:

4 → mouse

La probabilidad se convierte posteriormente a porcentaje para mostrarla en la interfaz.

12.8 Mostrar el resultado

Finalmente, JavaScript actualiza los elementos de la página:

```
resultIcon.textContent =
    EMOJIS[detectedClass] || "🔍";

resultName.textContent =
    detectedClass;

confidenceText.textContent =
    confidence.toFixed(2) + "%";

confidenceBar.style.width =
    Math.min(confidence, 100) + "%";
```

También se muestra un mensaje dependiendo del nivel de confianza:

```
if (confidence >= 80) {

    resultMessage.textContent =
        "El modelo reconoce este objeto con alta confianza.";

} else if (confidence >= 50) {

    resultMessage.textContent =
        "El modelo tiene una confianza moderada.";

} else {

    resultMessage.textContent =
        "El modelo no está completamente seguro del resultado.";

}
```

12.9 Eventos de la aplicación

Los botones se conectan con sus respectivas funciones mediante eventos:

```
startCameraButton.addEventListener(
    "click",
    startCamera
);

detectButton.addEventListener(
    "click",
    detectObject
);

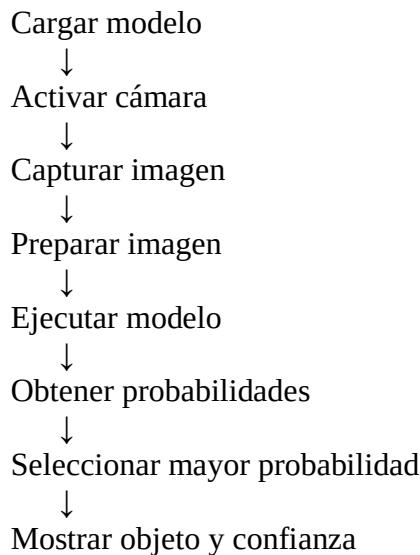
stopCameraButton.addEventListener(
    "click",
```

```
        stopCamera  
    );
```

Finalmente, se ejecuta `loadModel()` para cargar automáticamente el modelo cuando se inicia la aplicación:

```
loadModel();
```

De esta manera, el flujo principal de la aplicación queda establecido como:

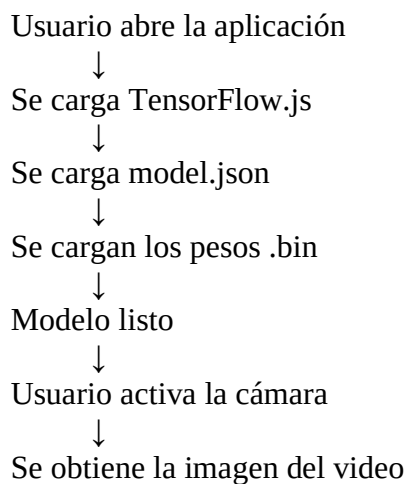


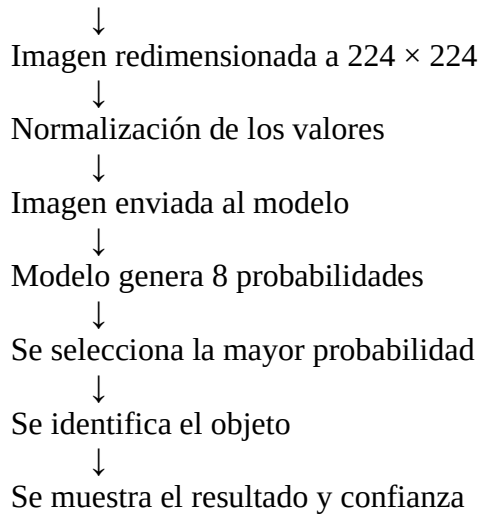
13. Funcionamiento general del sistema

El funcionamiento de **El Detector Mágico** se basa en la integración del modelo de Inteligencia Artificial con la aplicación web. El proceso comienza cuando el usuario abre la página y finaliza cuando se muestra el objeto detectado.

13.1 Flujo del sistema

El proceso general es el siguiente:





13.2 Integración de los componentes

Los principales componentes trabajan de la siguiente manera:

Componente	Función
index.html	Define la estructura de la aplicación.
style.css	Controla el diseño y apariencia.
app.js	Ejecuta la lógica de la aplicación.
TensorFlow.js	Permite ejecutar el modelo en el navegador.
model.json	Contiene la estructura del modelo.
Archivos .bin	Contienen los pesos del modelo.
Cámara web	Proporciona las imágenes para realizar las detecciones.

13.3 Proceso de clasificación

Cuando se presiona "**Detectar objeto**", la aplicación obtiene una imagen del video y la adapta al formato requerido por el modelo.

El modelo genera ocho valores, correspondientes a las ocho categorías:

- 0 → caja
- 1 → celular
- 2 → lentes
- 3 → manzana
- 4 → mouse
- 5 → platano
- 6 → reloj
- 7 → tijeras

El sistema compara estos valores y selecciona el que tenga la mayor probabilidad.

Por ejemplo:

caja → 2.10%
celular → 1.20%
lentes → 0.80%
manzana → 0.30%
mouse → 94.50%
platano → 0.20%
reloj → 0.60%
tijeras → 0.30%

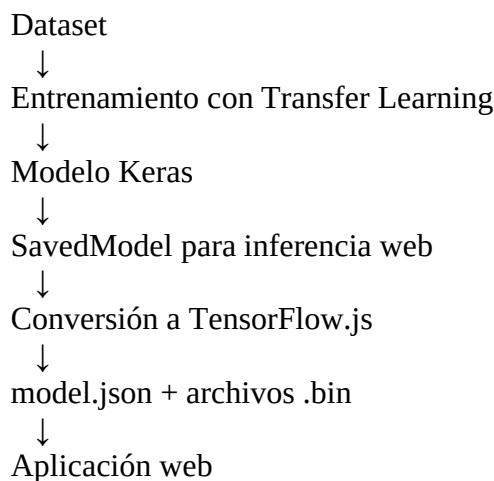
En este caso, el resultado sería:

Objeto detectado: Mouse
Confianza: 94.50%

13.4 Comunicación entre entrenamiento y aplicación

El modelo utilizado en la aplicación web corresponde al modelo entrenado previamente con las ocho categorías.

El proceso completo utilizado en el proyecto fue:



Esto permite que el modelo entrenado en Google Colab pueda utilizarse posteriormente desde el navegador sin necesidad de ejecutar Python o TensorFlow en el computador del usuario.

14. Pruebas y resultados

Para comprobar el funcionamiento del sistema se realizaron diferentes pruebas sobre el modelo de Inteligencia Artificial y la aplicación web.

14.1 Evaluación del modelo

El modelo fue evaluado utilizando el conjunto de datos de prueba compuesto por **360 imágenes correspondientes a las 8 categorías**.

El resultado obtenido fue:

Métrica	Resultado
Loss	0.0417
Accuracy	99.44 %

Este resultado indica que el modelo logró clasificar correctamente la mayoría de las imágenes utilizadas durante la evaluación.

14.2 Prueba individual de las categorías

También se realizó una prueba utilizando una imagen de cada una de las ocho categorías.

Los resultados fueron:

Categoría	Resultado
Caja	Correcto
Celular	Correcto
Lentes	Correcto
Manzana	Correcto
Mouse	Correcto
Plátano	Correcto
Reloj	Correcto
Tijeras	Correcto

Las ocho imágenes fueron clasificadas correctamente por el modelo, obteniendo porcentajes de confianza elevados.

14.3 Prueba de integración web

Finalmente, el modelo convertido a TensorFlow.js fue integrado en la aplicación web. Se comprobó que:

- El archivo model.json se carga correctamente.
- Los archivos .bin contienen los pesos necesarios para ejecutar el modelo.
- La cámara puede activarse desde el navegador.
- Las imágenes obtenidas desde la cámara son procesadas correctamente.
- El sistema muestra la categoría detectada y su porcentaje de confianza.

La prueba permitió verificar que el modelo entrenado puede utilizarse correctamente dentro de una aplicación web mediante TensorFlow.js.

15. Conclusiones

El desarrollo de **El Detector Mágico** permitió integrar Inteligencia Artificial y tecnologías web para crear una aplicación capaz de reconocer objetos mediante la cámara del dispositivo. El uso de **Transfer Learning con MobileNetV2** permitió obtener un modelo eficiente para la clasificación de las ocho categorías definidas.

Durante las pruebas, el modelo alcanzó una precisión de **99.44 % en el conjunto de datos de prueba** y clasificó correctamente las ocho categorías utilizadas en la prueba individual. Posteriormente, el modelo fue convertido a **TensorFlow.js** e integrado correctamente en la aplicación web.

Finalmente, la implementación de HTML, CSS, JavaScript y TensorFlow.js permitió construir una interfaz sencilla e interactiva, donde el usuario puede activar la cámara, realizar una detección y visualizar el objeto identificado junto con su nivel de confianza.